

TP – Algorithme Génétique pour Humans vs Zombies

Djebabla Ammar

Objectifs

- Comprendre le fonctionnement d'un **algorithme génétique** (GA) en Python.
- Appliquer un GA pour faire évoluer les caractéristiques des humains dans un jeu **Humans vs Zombies**.
- Travailler la programmation orientée objet et les structures de données.

1. Introduction

Dans le jeu **Humans vs Zombies**, chaque humain possède un **chromosome** qui définit sa vitesse, vision, précision, capacité de munitions et agressivité. L'objectif est d'implémenter une classe **GeneticAlgorithm** qui génère de nouvelles générations d'humains plus performants en fonction de leur **fitness**.

2. Structure de la classe à implémenter

Vous devez compléter la classe suivante :

```
class GeneticAlgorithm:
    def __init__(self, population_size=20):
        self.population_size = population_size
        self.mutation_rate = 0.25
        self.mutation_strength = 0.4
        self.elitism_count = 2 # Nombre de meilleurs individus
                               # conserver
        self.generation_stats = []

    def calculate_fitness(self, human):
        """
        Calculer le fitness d'un humain.
        Fitness plus sophistiquée :
        combiner kills, temps de survie, dégâts infligés, etc.
        """
        return human.fitness

    def selection(self, humans):
        """
        Sélection par rang :
        - Trier les humains par fitness
        - Garder les meilleurs
```

```

- Retourner une liste d'individus sélectionnés pour
  reproduction
"""
humans_sorted = sorted([h for h in humans if h.alive],
                        key=lambda h: h.fitness, reverse=True)
selected = []
# TODO : compléter la sélection par rang
return selected

def crossover(self, parent1, parent2):
    """
    Crossover deux points :
    - Choisir deux points aléatoires dans le chromosome
    - changer les gènes entre parent1 et parent2
    - Retourner le nouvel enfant
    """
    child = []
    # TODO : compléter le crossover
    return child

def mutate(self, chromosome, generation):
    """
    Mutation adaptative :
    - Plus la génération augmente, moins la mutation est forte
    """
    adaptive_rate = self.mutation_rate * (1.0 - min(generation / 50,
                                                    0.5))
    mutated = chromosome.copy()
    # TODO : appliquer mutation sur chaque gène
    return mutated

def create_new_generation(self, humans, generation):
    """
    Générer une nouvelle génération :
    - Conserver les meilleurs (elitism)
    - Sélectionner parents
    - Faire crossover
    - Appliquer mutation
    - Retourner la nouvelle population
    """
    new_population = []
    # TODO : implémenter la création de nouvelle génération
    return new_population

```

3. Exercices pratiques

Exercice 1 – Calcul du fitness

- Implémentez `calculate_fitness` en combinant :
 - Nombre de zombies tués
 - Temps de survie
 - Dégâts infligés
- Normalisez le score pour que le fitness soit entre 0 et 100.

Exercice 2 – Sélection par rang

- Trier les humains par fitness.
- Garder les `elitism_count` meilleurs directement.
- Remplir le reste de la sélection pour la reproduction.

Exercice 3 – Crossover

- Implémenter un crossover à deux points pour générer un enfant à partir de deux parents.
- Assurez-vous que le chromosome résultant a la même taille.

Exercice 4 – Mutation adaptative

- Implémenter une mutation aléatoire pour chaque gène du chromosome.
- Réduire l'intensité de mutation au fil des générations.

Exercice 5 – Nouvelle génération

- Conserver les meilleurs individus.
- Sélectionner parents et créer enfants par crossover.
- Appliquer mutation.
- Retourner la nouvelle population prête à jouer.

4. Bonus / Challenges

- Ajouter un suivi des statistiques par génération (moyenne, max, min du fitness).
- Visualiser l'évolution des humains dans le jeu en temps réel.
- Expérimenter différents taux de mutation et tailles de population.